scopes

- In simple terms, scope of a variable is its lifetime in the program.

- scope of a variable is the block of code in the entire program where the variable is declared, used, and can be modified.
-

```
#include <stdio.h>

int main()
{
int my_num = 7;
{
//add 10 my_num
my_num = my_num +10;

printf("my_num is %d",my_num);
}

return 0;
}
```

- **The main() function has an integer variable my_num that's initialized to 7 in the outer block.**
- **There's an inner block that tries to add 10 to the variable my_num.**
- **Now, compile and run the above program. Here's the output:**

- **//Output my_num is 17**

```c
#include <stdio.h>

int main()
{
   int my_num = 7;
   {
      int new_num = 10;
   }
   printf("new_num is %d",new_num);
   return 0;
}
```

- **In this program, the main() function has an integer variable my_num in the outer block.**
- **Another variable new_num is initialized in the inner block. The inner block is nested inside the outer block.**
- **We're trying to access and print the value of inner block's new_num in the outer block.**

If you try compiling that code, you'll notice that it doesn't compile successfully. And you'll get the following error message:

Line Message

9 error: 'new_num' undeclared (first use in this function)

reason-

This is because the variable new_num is declared in the inner block and its scope is limited to the inner block. In other words, it is local to the inner block and cannot be accessed from the outer block.

```
#include <stdio.h>

int main()
{
   int my_num = 7;
   printf("%d",my_num);
   my_func();
   return 0;
}

void my_func()
{
   printf("%d",my_num);
}
```

- The integer variable my_num is declared inside the main() function.
- Inside the main() function, the value of my_num is printed out.
- There's another function my_func() that tries to access and print the value of my_num.
- As program execution starts with the main() function, there's a call to my_func() inside the main() function.

- Now compile and run the above program. You'll get the following error message:
- Line Message error: 'my_num' undeclared (first use in this function)
- If you notice, on line 13, the function my_func() tried accessing the my_num variable that was declared and initialized inside the main() function.
- Therefore, the scope of the variable my_num is confined to the main() function, and is said to be local to the main() function.

Global Scope of Variables in C

```
#include <stdio.h>
int my_num = 7;

int main()
{
   printf("my_num can be accessed from main() and its value is %d\n",my_num);
   //call my_func
   my_func();
   return 0;
}

void my_func()
{
  printf("my_num can be accessed from my_func() as well and its value is %d\n",my_num);
}
```

- The variable my_num is declared outside the functions main() and my_func().
- We try to access my_num inside the main() function, and print its value.
- We call the function my_func() inside the main() function.
- The function my_func() also tries to access the value of my_num, and print it out.

//Output

my_num can be accessed from main() and its value is 7

my_num can be accessed from my_func() as well and its value is 7

the variable my_num is not local to any function in the program. Such a variable that is not local to any function is said to have global scope and is called a global variable.

```
/all global variables are declared here
function1()
   {

   // all global variables can be accessed inside function1

   }
function2()
   {

   // all global variables can be accessed inside function2

   }
```

- **The scope of a variable in C is the block or the region in the program where a variable is declared, defined, and used. Outside this region, we cannot access the variable, and it is treated as an undeclared identifier.**
- **The scope is the area under which a variable is visible.**
- **The scope of an identifier is the part of the program where the identifier may directly be accessible.**
- **We can only refer to a variable in its scope.**
- **In C, all identifiers are lexically(or statically) scoped.**

Static and Dynamic Scoping

- scope of a variable x in the region of the program in which the use of x refers to its declaration.
- One of the basic reasons for scoping is to keep variables in different parts of the program distinct from one another.

Static Scoping:

- Static scoping is also called lexical scoping. In this scoping, a variable always refers to its top-level environment.
- This is a property of the program text and is unrelated to the run-time call stack. Static scoping also makes it much easier to make a modular code as a programmer can figure out the scope just by looking at the code.
- In contrast, dynamic scope requires the programmer to anticipate all possible dynamic contexts.
- In most programming languages including C, C++, and Java, variables are always statically (or lexically) scoped i.e., binding of a variable can be determined by program text and is independent of the run-time function call stack.

```
#include<stdio.h>
int x = 10;
int f()
{
  return x;
}
int g()
{
  int x = 20;
  return f();
}
int main()
{
 printf("%d", g());
 printf("\n");
 return 0;
}
```

Output :

10

- in static scoping the compiler first searches in the current block, then in global variables, then in successively smaller scopes.

Dynamic Scoping:

- In simpler terms, in dynamic scoping, the compiler first searches the current block and then successively all the calling functions.
- uncommon in modern languages.
-

```
int x = 10;
int f()
{
   return x;
}
)
int g()
{
   int x = 20;
   return f();
}

main()
{
  printf(g());
}
```

Output in a language that uses Dynamic Scoping :

20

tatic Vs Dynamic Scoping

- In most programming languages static scoping is dominant. This is simply because in static scoping it’s easy to reason about and understand just by looking at code. We can see what variables are in the scope just by looking at the text in the editor.
- Dynamic scoping does not care about how the code is written, but instead how it executes. Each time a new function is executed, a new scope is pushed onto the stack.
- Perl supports both dynamic and static scoping.

- **Consider the program below in a hypothetical programming language which allows global variables and a choice of static or dynamic scoping.**

```
int i;
 main()
{
   i = 10;
   call f();
}

procedure f()
{
   int i = 20;
   call g ();
}
procedure g()
{
   print i;
}
```

question - Let x be the value printed under static scoping and y be the value printed under dynamic scoping. Then, x and y are:

1. **x=10,y=20**
2. **x=20,y=10**
3. **x=10,y=10**
4. **x=20,y=20**

- Option A must be the answer.
- As, under static scoping:x=10(globali)
- under dynamic scoping:y=20(according to the sequence of calls,i.e

What will be the output of the following pseudo-code when parameters are passed by reference and dynamic scoping is assumed?

```
a = 3;
void n(x) { x = x * a; print (x); }
void m(y) { a = 1 ; a = y - a;  n(a); print (a); }
void main () { m(a); }
```

options ;

a)6,2

b)6,6

c)4,2

d)4,4

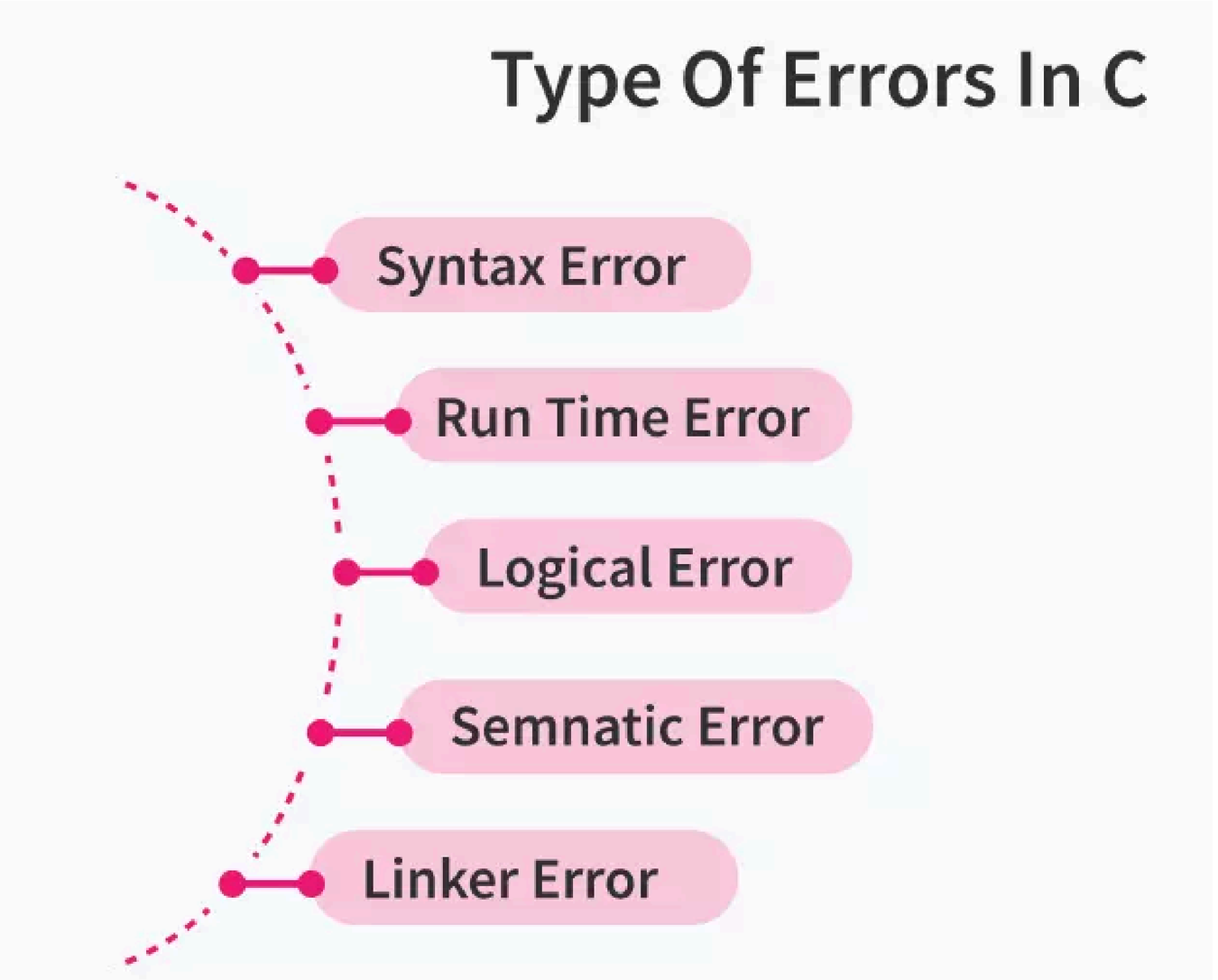
we can see that "a=3" and "a=1" are declaring new variables, one in global and other in local space.

Main is calling m(a). Since there is no local 'a', 'a' here is the global one.

In m, we have "a=1" which declares a local "a" and gives 1 to it. "a=y−a" assigns 3−1=2 to 'a'.

Now, in n(x), 'a' is used and as per dynamic scoping this 'a' comes from 'm()' and not the global one. So, "x=x∗a" assigns "2∗2=4" to "x" and 4 is printed. Being passed by reference, "a" in m() also get updated to 4. So, D is the answer here.

types of error in c



Syntax Error

- Syntax errors occur when a programmer makes mistakes in typing the code's syntax correctly or makes typos. In other words, syntax errors occur when a programmer does not follow the set of rules defined for the syntax of C language.
- Syntax errors are sometimes also called compilation errors because they are always detected by the compiler. Generally, these errors can be easily identified and rectified by programmers.

- **The most commonly occurring syntax errors in C language are:**
- **Missing semi-colon (;)**
- **Missing parenthesis ({})**
- **Assigning value to a variable without declaring it**

```
#include <stdio.h>

void main() {
    var = 5;    // we did not declare the data type of variable

    printf("The variable is: %d", var);
}
```

Output:

error: 'var' undeclared (first use in this function)

```
#include <stdio.h>

void main() {

    for (int i = 0;) {  // incorrect syntax of the for loop
        printf("Scaler Academy");
    }
}
```

Output:

error: expected expression before ')' token

A for loop needs 3 arguments to run. Since we entered only one argument, the compiler threw a syntax error.

Semantic Error

- Errors that occur because the compiler is unable to understand the written code are called Semantic Errors.
- A semantic error will be generated if the code makes no sense to the compiler, even though it is syntactically correct. It is like using the wrong word in the wrong place in the English language. For example, adding a string to an integer will generate a semantic error.
- Semantic errors are different from syntax errors, as syntax errors signify that the structure of a program is incorrect without considering its meaning. On the other hand, semantic errors signify the incorrect implementation of a program by considering the meaning of the program.
- The most commonly occurring semantic errors are: **use of un-initialized variables, type compatibility, and array index out of bounds.**

```
#include <stdio.h>

void main() {
   int a, b, c;

   a * b = c;

}
```

Output:

error: lvalue required as left operand of assignment

When we have an expression on the left-hand side of an assignment operator (=), the program generates a semantic error. Even though the code is syntactically correct, the compiler does not understand the code.

```
#include <stdio.h>

void main() {
    int arr[5] = {5, 10, 15, 20, 25};

    int arraySize = sizeof(arr)/sizeof(arr[0]);

    for (int i = 0; i <= arraySize; i++)
    {
        printf("%d \n", arr[i]);
    }
}
```

Output:
5
10
15
20
25
32764

In the example, we printed six elements while the array arr only had five. Because we tried to access the sixth element of the array, we got a semantic error and hence, the program generated a garbage value.

- **If there is a semantic error in your program, it will run successfully in the sense that the computer will not generate any error messages. However, your program will not do the right thing. It will do something else. Specifically, it will do what you told it to do, not what you wanted it to do.**
- **A semantic error is more like something that compiles fine (down to the very types), but isn't what you want it to be. Semantic errors are part of your algorithm more than your actual syntax.**

- **Syntax errors will cause a program to crash or not run at all. The program. may run until it encounters a syntax error, then it will stop.**

Runtime Error

- Errors that occur during the execution (or running) of a program are called RunTime Errors.
- These errors occur after the program has been compiled successfully. When a program is running, and it is not able to perform any particular operation, it means that we have encountered a run time error. For example, while a certain program is running, if it encounters the square root of -1 in the code, the program will not be able to generate an output because calculating the square root of -1 is not possible. Hence, the program will produce an error.
- Runtime errors can be a little tricky to identify because the compiler can not detect these errors.
- They can only be identified once the program is running. Some of the most common run time errors are: number not divisible by zero, array index out of bounds, string index out of bounds, etc.